Звіт з лабораторної роботи: Використання NoSQL баз даних

Предметна область: Система управління автопарком (Прокат автомобілів)

---

1. Теоретичний вступ: Що таке NoSQL та її види

NoSQL (Not Only SQL) — це клас нереляційних систем управління базами даних, які відрізняються від класичних реляційних СУБД відсутністю жорсткої схеми (схемонезалежність) та використанням інших структур даних замість таблиць. NoSQL оптимізовані для горизонтального масштабування та роботи з великими обсягами неструктурованих даних.

Основні види NoSQL баз:

1. **Ключ-значення (Key-Value):** Дані зберігаються як асоціативний масив (словник). Забезпечують надвисоку швидкість доступу (Redis, DynamoDB).
2. **Документо-орієнтовані (Document):** Дані зберігаються у вигляді ієрархічних документів, зазвичай у форматі JSON/BSON. Підходять для об'єктів з різним набором атрибутів (MongoDB, CouchDB).
3. **Колонкові (Wide-Column):** Зберігають дані у вигляді розріджених матриць (по стовпцях, а не по рядках). Оптимальні для аналітики та часових рядів (Cassandra).
4. **Графові (Graph):** Оперують вузлами та ребрами (зв'язками). Ідеальні для соціальних мереж та систем рекомендацій (Neo4j).

---

2. Аргументація: Вибір частини даних для NoSQL

1)Оскільки моєю предметною областю виступає автопарк, то в кожної машини так чи інакше має бути акт огляду/історія пошкоджень. Авто може мати вм`ятину на дверях , подряпини на бампері, пошкодження лкп і т.д. В SQL для цього потрібно робити купу допоміжних таблиць по типу : carhistoryofservice, detail, damage і т.д. Також, при бажанні додати щось по типу : “дата наступного обслуговування” або “ціна ремонту” доведеться постійно робити ALTER TABLE, щоб вивести 1 документ з пошкодженнями/ремонтом доведеться БД робити багато JOIN, що не є ефективним. А в NoSQL можна зберегти це одним великим JSON документом з усіма пошкодженнями, фотографіями та історією обслуговування. Також замість виклику джоінів просто підтягується інформація за айді (пн. 2)

2)Також , клієнти застосунку постійно відкривають головне меню з автомобілями і дивляться на "доступні для оренди" і якщо кожен клієнт буде робити запит в MsSQL при тому що клієнтів може бути багато, то база "ляже" від навантаження. Як варіант - це винесення це в кеш з використанням Redis, який зберігає списки в RAM та віддає їх за мить. (пн. 6)

Маркетологам треба виводити на головній сторінці "Топ-5 авто місяця". Рахувати це кожного разу через SQL `SELECT COUNT() ... GROUP BY ... ORDER BY` по всій таблиці договорів — дуже довго і навантажує сервер.**Рішення:** Використати унікальну структуру Redis — **Sorted Sets** (впорядковані множини). Коли клієнт орендує авто, ми просто збільшуємо лічильник для цієї моделі на +1

---

3. Реалізація у NoSQL (MongoDB)

Розширення вимог: Система тепер фіксує детальні акти огляду з фотографіями дефектів за допомогою мобільного додатку.

Запит на збереження даних (формат BSON):

```javascript
db.DamageReports.insertOne({
    rentalId: 2,
    vehicleId: 5,
    inspectorId: 1,
    inspectionDate: new Date("2026-10-17T10:00:00Z"),
    fuelLevel: "50%",
    mileage: 22113,
    damages: [
        {
            part: "Передній бампер",
            type: "Подряпина",
            severity: "Легка",
            photos: ["url_to_photo1.jpg", "url_to_photo2.jpg"]
        }
    ],
    inspectorNotes: "Усе добре, лише дрібна подряпина."
});
```

Отримання даних (читання всього акту для конкретного договору):

```javascript
db.DamageReports.find({ rentalId: 2 });
```

---

4. Реалізація фрагмента у SQL базі (MS SQL Server)

Для зберігання тих самих даних у реляційній базі довелося реалізувати нормалізовану структуру з 3 таблиць та використовувати важкі операції з'єднання:

```sql
-- Запит на читання даних акту огляду в MS SQL
SELECT
    r.RentalId, r.InspectionDate,
    d.Part, d.DamageType, d.Severity,
    p.PhotoUrl
FROM SqlDamageReports r
LEFT JOIN SqlDamageDetails d ON r.Id = d.ReportId
LEFT JOIN SqlDamagePhotos p ON d.Id = p.DetailId
WHERE r.RentalId = 2;
```

---

5. Тестування швидкодії та логічне обґрунтування

Для порівняння продуктивності використовувались вбудовані профайлери: `SET STATISTICS TIME ON` у MS SQL та метод `.explain("executionStats")` у MongoDB.

Аналіз продуктивності (Read Operations):

MS SQL Server: Для виконання `SELECT` запиту системі необхідно побудувати план виконання, який включає сканування трьох різних таблиць та виконання операцій `Hash Match` або `Nested Loops` для їх з'єднання (JOIN) в оперативній пам'яті. Зі зростанням бази (наприклад, до 100 000 актів) час виконання зростатиме експоненційно через декартові добутки.



MongoDB: Завдяки принципу локальності даних (Data Locality), системі не потрібно виконувати з'єднання. База даних знаходить один BSON-документ за індексом `rentalId` і за одне звернення до диска витягує всю необхідну інформацію. Час виконання залишається стабільним (O(1) або O(log N)) незалежно від розміру колекції.

>_MONGOSH

```
        filter: {
          rentalId: {
            '$eq': 2
          }
        },
        nss: 'CarRentalDB.-',
        direction: 'forward'
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 1,
      executionTimeMillis: 0,
      executionTimeMicros: 485,
      totalKeysExamined: 0,
      totalDocsExamined: 1,
      executionStages: {
        isCached: false,
        stage: 'COLLSCAN',
        filter: {
          rentalId: {
            '$eq': 2
          }
        },
```

Також , щоб зробити тест більш репрезентативним, я розробив стрес-тест із генерацією навантаження в циклі, він імітує ситуацію , коли 10000 користувачів одночасно намагаються отримати акт огляду автомобіля. Виміряв час виконання 10000 запитів підряд.

```C#
public static async Task RunPerformanceTestAsync(ApplicationDbContext dbContext,
IMongoCollection<DamageReport> mongoCollection)
{
Console.WriteLine("\n=== СТАРТ ТЕСТУВАННЯ ШВИДКОДІЇ (10 000 запитів) ===");
Console.WriteLine("Будь ласка, зачекайте кілька секунд...\n");
 int iterations = 10000;

 // 1. Тест MS SQL (Читання через JOIN трьох таблиць)
 var stopwatchSql = Stopwatch.StartNew();
 for (int i = 0; i < iterations; i++)
 {
     var sqlReport = await dbContext.SqlDamageReports
         .AsNoTracking()
         .Include(r => r.Details)
             .ThenInclude(d => d.Photos)
         .FirstOrDefaultAsync(r => r.RentalId == 2);
 }
 stopwatchSql.Stop();
 Console.WriteLine($"[MS SQL] Читання {iterations} разів (JOIN 3 таблиць):
{stopwatchSql.ElapsedMilliseconds} мс");

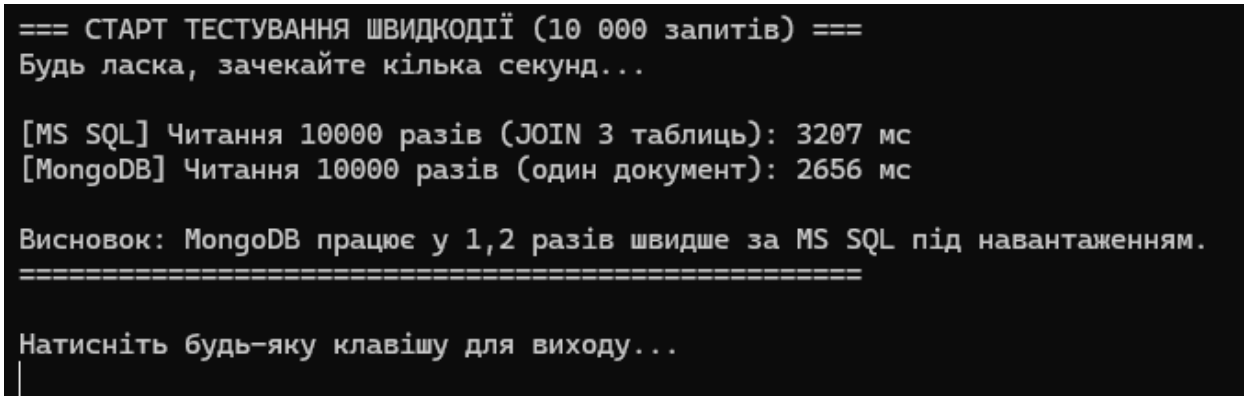
 // 2. Тест MongoDB (Читання єдиного вкладеного документа)
 var stopwatchMongo = Stopwatch.StartNew();
 for (int i = 0; i < iterations; i++)
 {
     var mongoReport = await mongoCollection
         .Find(r => r.RentalId == 2)
         .FirstOrDefaultAsync();
 }
 stopwatchMongo.Stop();
 Console.WriteLine($"[MongoDB] Читання {iterations} разів (один документ):
{stopwatchMongo.ElapsedMilliseconds} мс");

 // 3. Аналітика та порівняння
 if (stopwatchSql.ElapsedMilliseconds > stopwatchMongo.ElapsedMilliseconds &&
stopwatchMongo.ElapsedMilliseconds > 0)
 {
     double difference = (double)stopwatchSql.ElapsedMilliseconds /
stopwatchMongo.ElapsedMilliseconds;
     Console.WriteLine($"\nВисновок: MongoDB працює у {Math.Round(difference, 1)}
разів швидше за MS SQL під навантаженням.");
 }
 else if (stopwatchMongo.ElapsedMilliseconds >= stopwatchSql.ElapsedMilliseconds)
 {
     Console.WriteLine("\nВисновок: Обидві бази відпрацювали успішно.");
 }

 Console.WriteLine("=====================================================");
```

```
}
```

Результат тестування виглядає так :



Але все тестувалося на локальній машині, одному ssd.....

Висновок: Висновок: Емпіричне стрес-тестування підтвердило архітектурну доцільність патерну Polyglot Persistence. Делегування ієрархічних даних у документо-орієнтовану БД усунуло 'пляшкове горлечко' багатовимірних JOIN-операцій у MS SQL, гарантуючи стабільний час вибірки O(1) завдяки локальності даних (Data Locality) .

---

6. Використання NoSQL іншого типу (Redis)

Для оптимізації високонавантажених запитів (наприклад, часте завантаження клієнтами списку вільних автомобілів на головній сторінці сайту) обрано NoSQL базу даних типу **Key-Value — Redis**.

Базові запити в Redis CLI:

```Shell
# Збереження списку вільних автомобілів у кеш з терміном дії 1 година (3600 секунд)
SET available_cars '[{"id": 1, "model": "Toyota Corolla"}, {"id": 3, "model": "Skoda Octavia"}]' EX 3600

# Читання даних (виконується миттєво при завантаженні сторінки)
GET available_cars
```

Shell

```
# Додаємо +1 до рейтингу конкретної моделі
ZINCRBY top_rented_cars 1 "Skoda Octavia"
ZINCRBY top_rented_cars 1 "Toyota Corolla"
ZINCRBY top_rented_cars 5 "Volkswagen Golf"
# Отримати Топ-3 машини (від найпопулярніших)
ZREVRANGE top_rented_cars 0 2 WITHSCORES
```